

In[1061]:=

```
ClearAll["Global`*"];
|清除全部

SeedRandom[42];
|随机种子

(* ===== *)
(* TCCT 拓扑异或门测试台 (Topological XOR Gate TestBench) *)
(* 核心机制:利用并发拥堵与 Union 矢量去重产生局部短路 *)
|并集

(* ===== *)

(*1. 定义真空脚手架与基础引擎*)
xorNodes = {"A", "B", "M", "C"};
|常量

baseEdges = {UndirectedEdge["M", "C"]}; (*M到C的预置通道, 等待被激活*)
|无向边 |常量

vacuumGraph = Graph[xorNodes, baseEdges, VertexLabels -> Placed["Name", Center],
|图 |顶点的标签 |放置 |居中
    VertexSize -> 0.3, GraphLayout -> "SpringEmbedding"];
|顶点大小 |图的布局

(*信号注入探针 (编码输入态)*)
InjectXOR[inputA_, inputB_] := Module[{g = vacuumGraph},
|模块

    (*注入拓扑势能 (活性火种) *) If[inputA == 1, g = EdgeAdd[g, UndirectedEdge["A", "M"]]];
|如果 |添加边 |无向边

    If[inputB == 1, g = EdgeAdd[g, UndirectedEdge["B", "M"]]];
|如果 |添加边 |无向边

    Return[g];
|返回

(*2. 微观演化引擎 (波前相消优先+绝对 Union 去重) *)
|并集

StepEvolutionXOR[g_Graph] :=
|图

Module[{allEdges, activeEdges, pairs, selectedPair, e1, e2, x,
|模块

    y, z, w, intNodes, newActive, newFrozen}, allEdges = EdgeList[g];
|边的列表

    activeEdges = Cases[allEdges, _UndirectedEdge];
|模式匹配 |无向边

    (*只有无向边可以参与反应*) If[Length[activeEdges] < 2, Return[g]];
|... |长度 |返回

    (*寻找所有合法的因果干涉对 (共享一个节点的两条活性边) *) pairs = Select[
|选择

        Subsets[activeEdges, {2}], Length[Intersection[List @@ #[[1]], List @@ #[[2]]]] == 1 &];
|子集 |长度 |交集 |列表 |列表

    If[Length[pairs] == 0, Return[g]];
|... |长度 |返回

    (*【物理相干补丁】: 模拟双狭缝并发波前, 强制同时抵达的波峰优先相互碰撞*)
```

```

selectedPair = First[pairs];
|第一个
Do[If[ (MemberQ[p, UndirectedEdge["A", "M"]] || MemberQ[p, UndirectedEdge["M", "A"]]) &&
|… |如果 |成员判定 |无向边 |成员判定 |无向边
  (MemberQ[p, UndirectedEdge["B", "M"]] ||
  |成员判定 |无向边
    MemberQ[p, UndirectedEdge["M", "B"]]), selectedPair = p;
  |成员判定 |无向边
  Break[];], {p, pairs}];
|跳出循环
e1 = selectedPair[[1]];
e2 = selectedPair[[2]];
y = Intersection[List @@ e1, List @@ e2][[1]];
|交集 |列表 |列表
x = Complement[List @@ e1, {y}][[1]];
|补集 |列表
z = Complement[List @@ e2, {y}][[1]];
|补集 |列表
(*创生新拓扑维度节点 w*) intNodes = Cases[VertexList[g], _Integer];
|模… |顶点列表 |整数
w = If[Length[intNodes] == 0, 1, Max[intNodes] + 1];
|… |长度 |最大值
(*波函数坍缩 (RAM→Disk):反应物被冻结为有向历史边*)
|圆盘
newFrozen = {DirectedEdge[x, y], DirectedEdge[y, z]};
|有向边 |有向边
(*活性暴胀:生成全新的 Tri-Chord 活性边*)
newActive = {UndirectedEdge[x, z], UndirectedEdge[x, w], UndirectedEdge[w, z]};
|无向边 |无向边 |无向边
(*核心! 更新宇宙网络状态: 使用 Union 强制执行布尔矢量去重 (1+1=1)*)
|并集
Return[Graph[Join[VertexList[g], {w}], Union[Complement[allEdges, {e1, e2}],
|返回 |图 |连接 |顶点列表 |并集 |补集
  newFrozen, newActive], (*绝对严谨的拓扑坍缩*) VertexLabels → Placed["Name", Center],
  |顶点的标签 |放置 |居中
  VertexSize → 0.3, (*渲染设定:活性势能边为青色粗线, 冻结历史边为灰色细线*)
  |顶点大小
  EdgeStyle → {_UndirectedEdge → Directive[Cyan, Thick],
  |边的样式 |无向边 |指令 |蓝绿色 |粗
    _DirectedEdge → Directive[Gray, Thin]}, Background → Black]]];
  |有向边 |指令 |灰色 |细 |背景色 |黑色

(*运行测试封装*)
RunXORTest[inputA_, inputB_, steps_] :=
Module[{g = InjectXOR[inputA, inputB], i}, Do[g = StepEvolutionXOR[g], {i, 1, steps}];
|模块 |Do循环
  Return[g];]
|返回

(*3. 终极测量算符 (Readout Collapse) *)
MeasureOutput[finalGraph_] :=
Module[{finalEdges, directedMC, newActiveToC}, finalEdges = EdgeList[finalGraph];
|模块 |边的列表

```

```

(*检查1: 预置边 M-C 是否被作为反应物消耗, 坍塌为了历史有向边? *)
|常量
directedMC = MemberQ[finalEdges, DirectedEdge["M", "C"]] ||
|成员判定 |有向边 |常量
MemberQ[finalEdges, DirectedEdge["C", "M"]];
|成员判定 |有向边 |常量
(*检查2: 输出节点 C 是否接收到了除 M 之外的、全新的活性火种? *)
|常量
newActiveToC = Length[Cases[finalEdges, UndirectedEdge["C", n_] /; n != "M"]] +
|长度 |模式匹配 |无向边 |常量
Length[Cases[finalEdges, UndirectedEdge[n_, "C"] /; n != "M"]];
|长度 |模式匹配 |无向边 |常量
(*物理判决: 必须同时满足上游因果消耗与新活性波抵达, 才算逻辑 1*)
If[directedMC && newActiveToC > 0, 1, 0]
|如果

(* ===== *)
(*4. 跑批与流形渲染 (Truth Table Execution)*)
|表格

(* ===== *)
Print[Style["=== TCCT Topological XOR Gate TestBench ===",
|打印 |样式
White, Bold, 16, Background -> Black]];
|白色 |粗体 |背景色 |黑色
testCases = {{0, 0}, {0, 1}, {1, 0}, {1, 1}};
results = {};
graphs = {};

Do[inA = testCases[[i, 1]];
|Do循环
inB = testCases[[i, 2]];
(*1步并发碰撞即可完成局部短路与相消干涉*) finalG = RunXORTest[inA, inB, 1];
outC = MeasureOutput[finalG];
AppendTo[results, {inA, inB, outC}];
|附加
AppendTo[graphs, finalG];
|附加
Print[Style[StringForm["Input (A:`, B:`) -> Output C: `", inA, inB, outC],
|打印 |样式 |字符串形式 |输入 |常量
14, Bold, If[outC == 1, Green, Red]]];, {i, 1, 4}];
|粗体 |如果 |绿色 |红色

(*渲染四组叠加态坍塌后的宏观拓扑图谱*)
Print[GraphicsGrid[Partition[MapThread[
|打印 |图形网格 |划分 |映射线程
Labeled[#1, Style[StringForm["(A:`, B:`) -> Output: `", #2[[1]], #2[[2]], #2[[3]]],
|标记 |样式 |字符串形式
14, Bold, White]] &, {graphs, results}], 2], ImageSize -> 800, Background -> Black]
|粗体 |白色 |图像尺寸 |背景色 |黑色

=== TCCT Topological XOR Gate TestBench ===

```

Input (A:0, B:0) -> Output C: 0

Input (A:0, B:1) -> Output C: 1

Input (A:1, B:0) -> Output C: 1

Input (A:1, B:1) -> Output C: 0

